Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Marco Castillo Viviana Córdova Isauro Rivera Ariel Tandazo
Asignatura	Sistemas Distribuidos
Ciclo	6 A
Unidad	1
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	005
Título de la Práctica	Implementar un servicio REST con Node.js
Nombre del Docente	José O. Guamán Q.
Fecha	20 de octubre de 2026 23 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Implementar la capa IoT de un sistema de monitoreo de flota logística utilizando el protocolo MQTT: configurar un broker Mosquitto en Docker, simular sensores que publican telemetría en topics jerárquicos con paho-mqtt, implementar un suscriptor que almacene los datos en SQLite y genere alertas por umbral, y construir un servicio puente que publique las alertas en colas de RabbitMQ.

Comprender los fundamentos del protocolo MQTT y su aplicacion en entornos IoT

- Configurar e instalar un broker Mosquitto para mensajeria publicador/suscriptor
- Implementar publicadores y suscriptores MQTT en Python usando paho-mqtt
- Disenar topic hierarchies para organizar la telemetria de una flota vehicular
- Entender el protocolo AMQP 0-9-1 y el modelo de intercambio de RabbitMQ
- Configurar exchanges, colas y bindings en RabbitMQ
- Implementar microservicios en Spring Boot que consuman mensajes de RabbitMQ
- Integrar MQTT y RabbitMQ mediante un servicio puente en Python
- Aplicar patrones de mensajería: pub/sub, routing y competing consumers.

3. Materiales, Reactivos, Equipos y Herramientas

Guia

practica

Software

- Python
- paho-mqtt
- pika
- Java JDK 17
- Spring Boot
- Docker
- Docker Compose
- Mosquito
- RabbitMQ
- SQLite3
- IDE
- Postman
- Navegador

web Hardware

- Computador

4. Procedimiento / Metodología Ejecutada

Fase 1: Despliegue de Infraestructura y Servidores en Contenedores(Docker)

Esta fase inicial establece el entorno de ejecución aislando los componentes clave del sistema dentro de contenedores Docker. Se utiliza Docker Compose para desplegar e interconectar en una misma red virtual el broker MQTT (Eclipse Mosquitto) para la captación ligera de telemetría y el broker empresarial (RabbitMQ) con su consola web de administración habilitada

Imagen contenedor activo (UP)

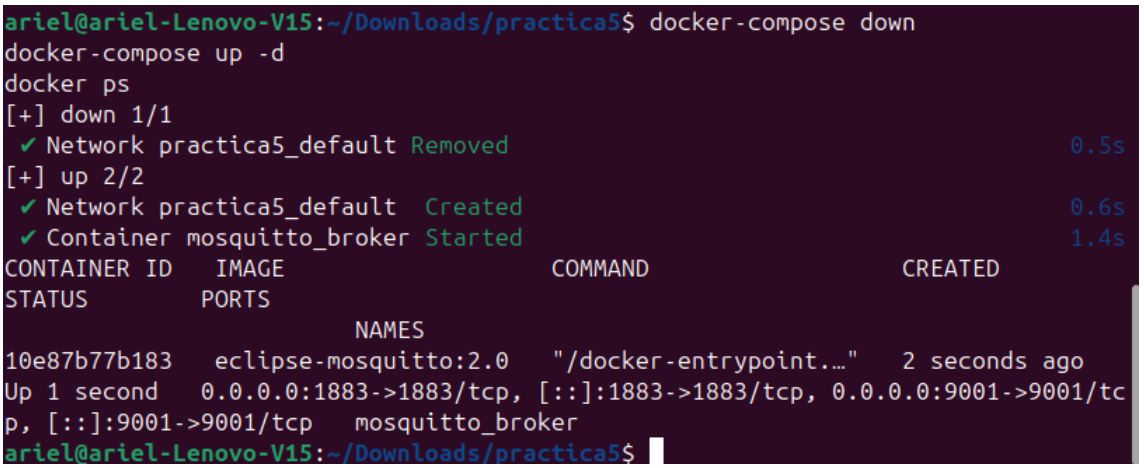
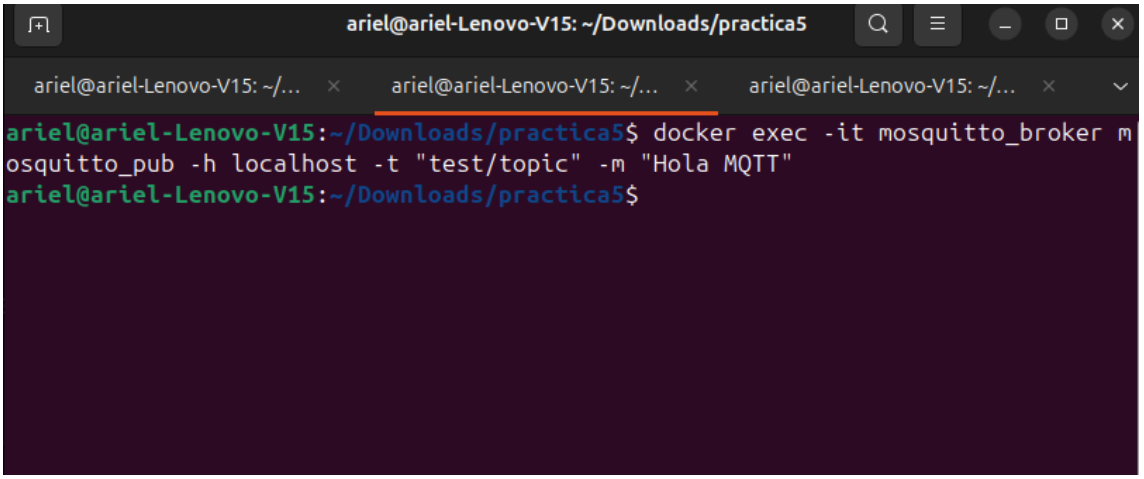


Imagen 1: contenedor activo (UP)

Código de Configuración: Permisos de Mosquitto (`mosquitto.conf`)



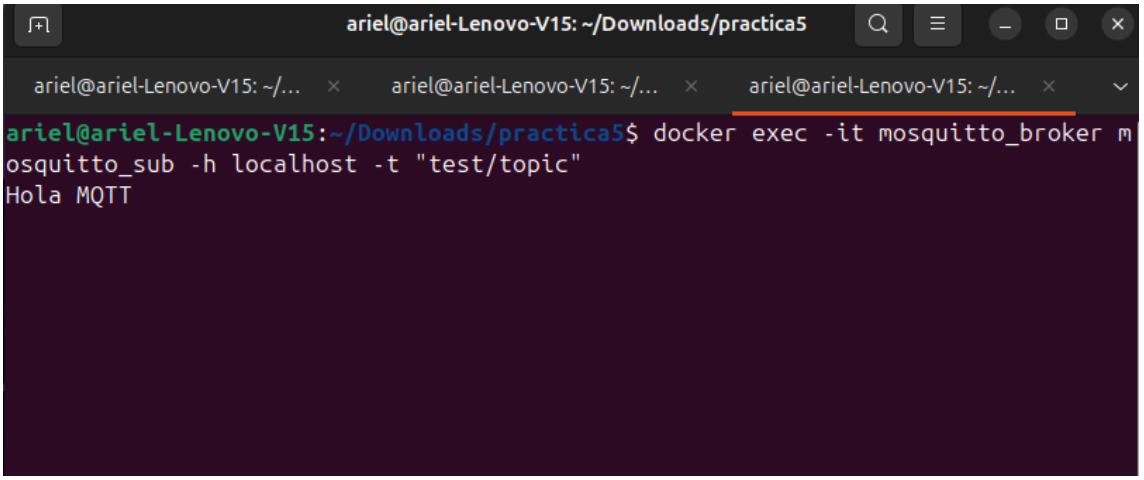


Imagen x:

Fase 2: Estructuración del Formato de Datos y Canales de Comunicación

Flota para los 3 sensores (VH-001, VH002, VH003):

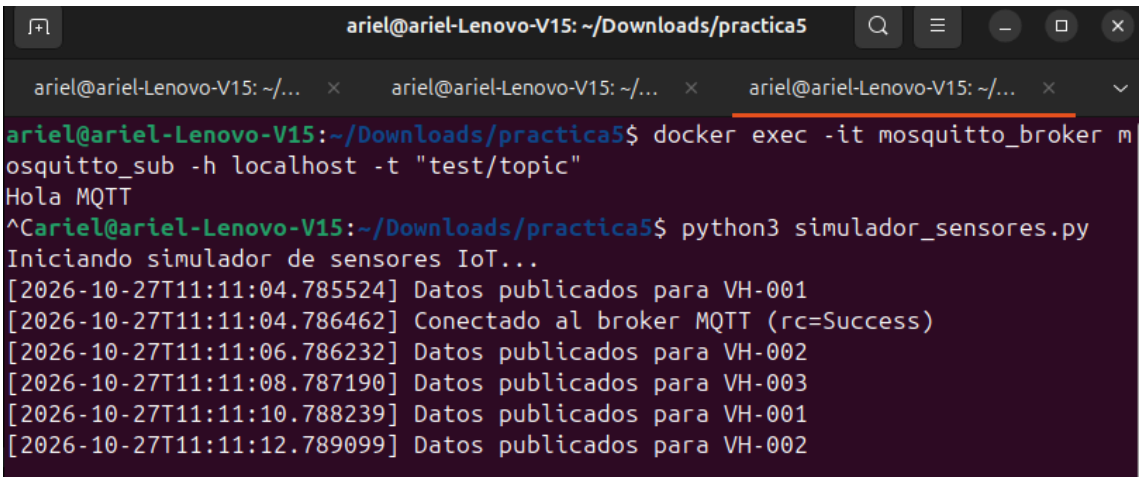


Imagen x:

Flota para los 3 sensores (VH-001, VH002, VH003):

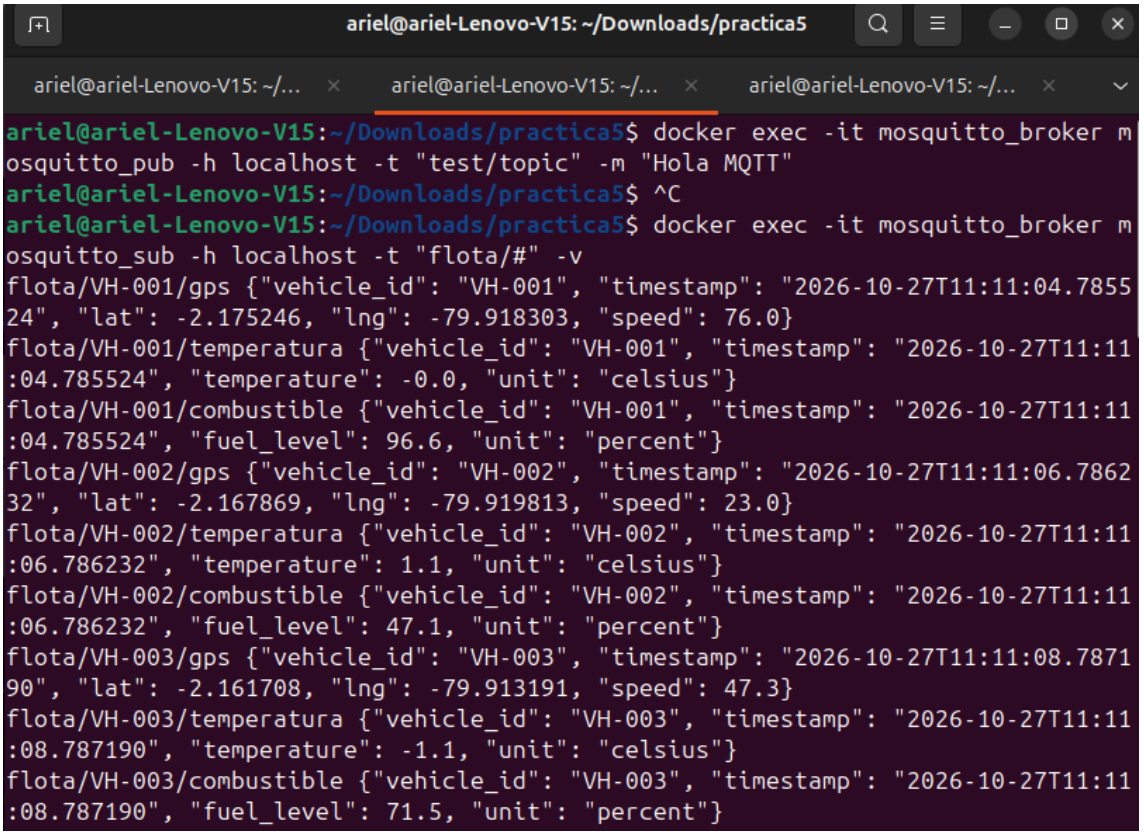


Imagen x:

Fase 2: Estructuración del Formato de Datos y Canales de Comunicación

Se formaliza el contrato de datos en JSON para la telemetría (ubicación GPS, temperatura de carga y nivel de combustible) y se define la taxonomía de tópicos MQTT que organiza los mensajes por región y por vehículo.

Esquema JSON de Telemetría

```
marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯ cat esquema_telemetria.json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://flota.local/esquema_telemetria.json",
  "title": "Telemetría de flota logística",
  "description": "Contrato JSON para las lecturas publicadas por los vehículos en MQTT.",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schema_version",
    "vehicle_id",
    "region",
    "sensor",
    "timestamp",
    "data"
  ],
  "properties": {
    "schema_version": {
      "type": "string",
      "const": "1.0"
    },
    "vehicle_id": {
      "type": "string",
      "pattern": "^VH-[0-9]{3}$",
      "examples": ["VH-001", "VH-002", "VH-003"]
    },
    "region": {
      "type": "string",
      "enum": ["guayaquil", "quito", "cuenca"]
    },
    "sensor": {
```

Validación de telemetría:

```
marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯ .venv/bin/python -c "import j
son; from jsonschema import Draft202012Validator as V; V(json.load(open('esquema_telemetria.json')
)).validate(json.load(open('capturas/fase2_esquema_ejemplo.json'))); print('Esquema de telemetria
valido: OK')"
Esquema de telemetria valido: OK

marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯
```

Fase 3: Simulación de Captura y Emisión IoT desde Vehículos

Desarrollo de la capa de borde (Edge Layer) mediante un script en Python que simula los sensores del camión. Publica datos periódicamente al contenedor de Mosquitto mediante Paho MQTT con calidad de servicio QoS 1.a

Captura: Simulador de Sensores IoT (`emisor_vehiculo.py`)

```
marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯ .venv/bin/python -u suscriptor_telemetria.py
[2026-10-30T08:25:55-05:00] Base de datos inicializada.
Umbral de temperatura: 5.0 °C
[2026-10-30T08:25:55-05:00] Conectado a Mosquitto; suscribiendo a flota/+/+/+
[2026-10-30T08:26:25-05:00] Guardado gps de VH-001 (guayaquil): lat=-2.166247 lng=-79.918725 speed=70.8
[2026-10-30T08:26:25-05:00] Guardado temperatura de VH-001 (guayaquil): temperature=5.5 celsius
[ALERTA] ADVERTENCIA VH-001 (guayaquil): 5.5 °C > 5.0 °C
[2026-10-30T08:26:25-05:00] Guardado combustible de VH-001 (guayaquil): fuel_level=88.9 percent
[2026-10-30T08:26:30-05:00] Guardado gps de VH-002 (quito): lat=-0.187068 lng=-78.47185 speed=40.1
[2026-10-30T08:26:30-05:00] Guardado temperatura de VH-002 (quito): temperature=1.9 celsius
[2026-10-30T08:26:30-05:00] Guardado combustible de VH-002 (quito): fuel_level=29.9 percent
[2026-10-30T08:26:35-05:00] Guardado gps de VH-003 (cuenca): lat=-2.907306 lng=-78.999898 speed=67.1
[2026-10-30T08:26:35-05:00] Guardado temperatura de VH-003 (cuenca): temperature=-4.2 celsius
[2026-10-30T08:26:35-05:00] Guardado combustible de VH-003 (cuenca): fuel_level=56.0 percent
[2026-10-30T08:26:40-05:00] Guardado gps de VH-001 (guayaquil): lat=-2.174823 lng=-79.919656 speed=49.6
[2026-10-30T08:26:40-05:00] Guardado temperatura de VH-001 (guayaquil): temperature=7.7 celsius
[ALERTA] CRITICA VH-001 (guayaquil): 7.7 °C > 5.0 °C
[2026-10-30T08:26:40-05:00] Guardado combustible de VH-001 (guayaquil): fuel_level=63.2 percent
[2026-10-30T08:26:45-05:00] Guardado gps de VH-002 (quito): lat=-0.18169 lng=-78.466003 speed=50.7
[2026-10-30T08:26:45-05:00] Guardado temperatura de VH-002 (quito): temperature=-1.3 celsius
[2026-10-30T08:26:45-05:00] Guardado combustible de VH-002 (quito): fuel_level=51.3 percent
[2026-10-30T08:26:50-05:00] Guardado gps de VH-003 (cuenca): lat=-2.900672 lng=-78.996882 speed=34.9
[2026-10-30T08:26:50-05:00] Guardado temperatura de VH-003 (cuenca): temperature=3.5 celsius
```

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://flota.local/esquema_alerta.json",
  "title": "Alerta de cadena de frío",
  "description": "Mensaje que publica el puente MQTT-AMQP cuando la temperatura supera el umbral."
,
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schema_version",
    "alert_id",
    "vehicle_id",
    "region",
    "sensor",
    "timestamp",
    "temperature",
    "threshold",
    "severity",
    "message",
    "source"
  ],
  "properties": {
    "schema_version": {"type": "string", "const": "1.0"},
    "alert_id": {"type": "string", "minLength": 1},
    "vehicle_id": {"type": "string", "pattern": "^VH-[0-9]{3}$"},
    "region": {"type": "string", "enum": ["guayaquil", "quito", "cuenca"]},
    "sensor": {"type": "string", "const": "temperatura"},
    "timestamp": {"type": "string", "format": "date-time"},
    "temperature": {"type": "number"},
    "threshold": {"type": "number"},
    "severity": {"type": "string", "enum": ["ADVERTENCIA", "CRITICA"]},
```

Consulta a la BD:

```
marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯ .venv/bin/python consultar_db.py --tabla alertas
--limite 5

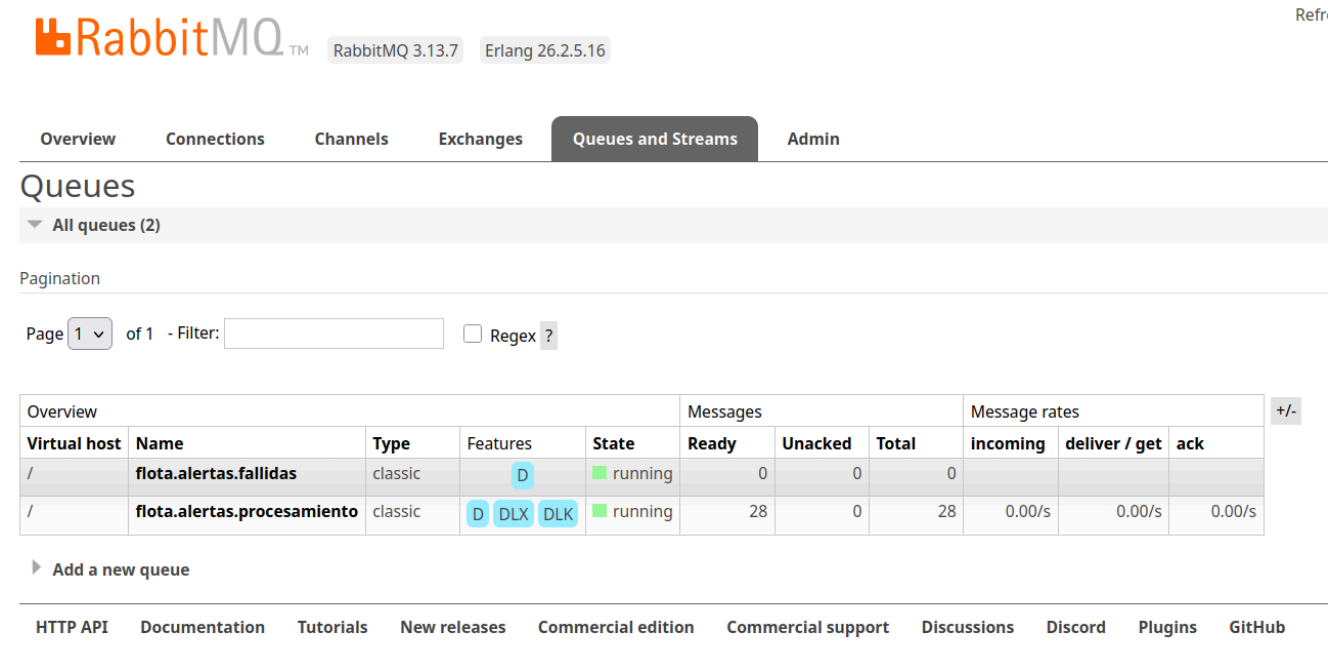
== alertas (2 filas) ==
id | alert_id | vehicle_id | region | timestamp | temperature | threshold | severity | message | received_at
--------------------------------------------------------------------------------
2 | 187eb2fd-59e1-48b0-8194-5a6015a55156 | VH-003 | cuenca | 2026-10-30T08:50:49-05:00 | 8.3 | 5.0 | CRITICA | Tempera
tura 8.3 °C supera el umbral de 5.0 °C en VH-003 (cuenca). | 2026-10-30T13:50:49+00:00
1 | fb33c967-b923-4979-8329-820a1b2bba60 | VH-001 | guayaquil | 2026-10-30T08:50:45-05:00 | 7.5 | 5.0 | CRITICA | Temp
eratura 7.5 °C supera el umbral de 5.0 °C en VH-001 (guayaquil). | 2026-10-30T13:50:45+00:00
```

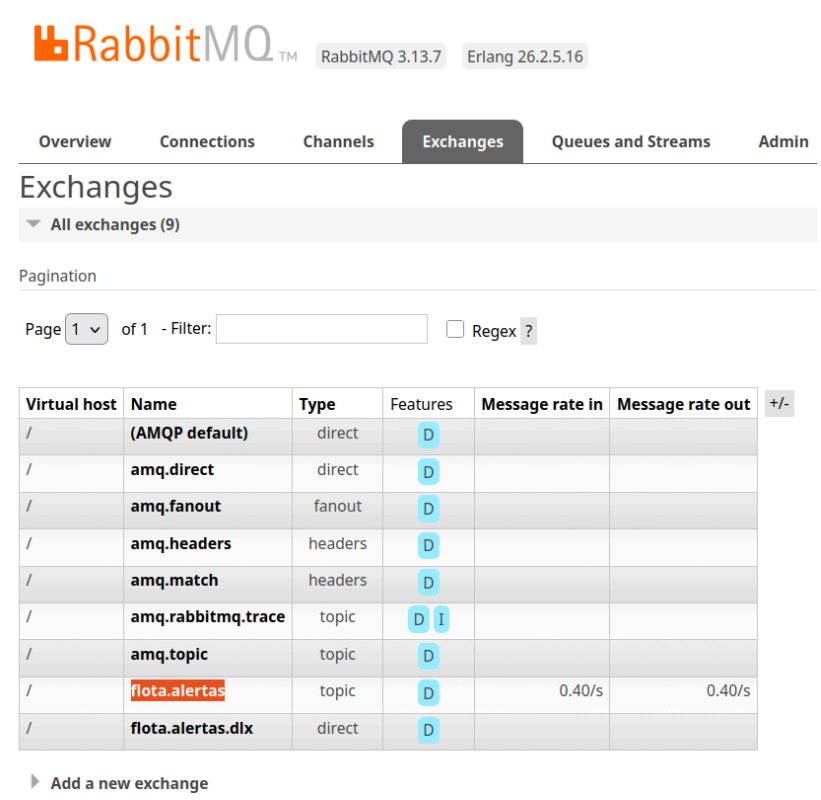
Fase 4: Construcción del Adaptador de Integración (Puente MQTT-AMQP)

Implementación del servicio puente que intercepta los datos del contenedor Mosquitto, transforma el paquete y lo retransmite hacia el Exchange del contenedor RabbitMQ.

Código de Desarrollo: Servicio Puente (`adaptador_puente.py`)

```
marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯ .venv/bin/python -u adaptador_puente.py
[2026-10-30T08:26:40-05:00] Conectado a RabbitMQ; topología declarada.
[2026-10-30T08:26:40-05:00] Puente MQTT-AMQP en ejecución.
[2026-10-30T08:26:40-05:00] Conectado a Mosquitto; suscribiendo a flota/+/+/temperatura
[2026-10-30T08:27:20-05:00] Alerta publicada en RabbitMQ (alerta.cuenca.VH-003): VH-003 6.8 °C
[2026-10-30T08:27:55-05:00] Alerta publicada en RabbitMQ (alerta.guayaquil.VH-001): VH-001 6.9 °C
[2026-10-30T08:28:00-05:00] Alerta publicada en RabbitMQ (alerta.quito.VH-002): VH-002 8.3 °C
[2026-10-30T08:28:10-05:00] Alerta publicada en RabbitMQ (alerta.guayaquil.VH-001): VH-001 5.8 °C
[2026-10-30T08:28:30-05:00] Alerta publicada en RabbitMQ (alerta.quito.VH-002): VH-002 7.8 °C
[2026-10-30T08:28:45-05:00] Alerta publicada en RabbitMQ (alerta.quito.VH-002): VH-002 7.7 °C
[2026-10-30T08:29:15-05:00] Alerta publicada en RabbitMQ (alerta.quito.VH-002): VH-002 7.3 °C
[2026-10-30T08:29:20-05:00] Alerta publicada en RabbitMQ (alerta.cuenca.VH-003): VH-003 6.9 °C
[2026-10-30T08:29:25-05:00] Alerta publicada en RabbitMQ (alerta.guayaquil.VH-001): VH-001 5.7 °C
[2026-10-30T08:29:30-05:00] Alerta publicada en RabbitMQ (alerta.quito.VH-002): VH-002 7.1 °C
[2026-10-30T08:29:45-05:00] Alerta publicada en RabbitMQ (alerta.quito.VH-002): VH-002 7.9 °C
[2026-10-30T08:29:50-05:00] Alerta publicada en RabbitMQ (alerta.cuenca.VH-003): VH-003 5.5 °C
[2026-10-30T08:29:56-05:00] Alerta publicada en RabbitMQ (alerta.guayaquil.VH-001): VH-001 8.3 °C
[2026-10-30T08:29:57-05:00] Alerta publicada en RabbitMQ (alerta.guayaquil.VH-001): VH-001 7.5 °C
[2026-10-30T08:30:03-05:00] Alerta publicada en RabbitMQ (alerta.guayaquil.VH-001): VH-001 8.6 °C
[2026-10-30T08:30:05-05:00] Alerta publicada en RabbitMQ (alerta.quito.VH-002): VH-002 7.6 °C
```



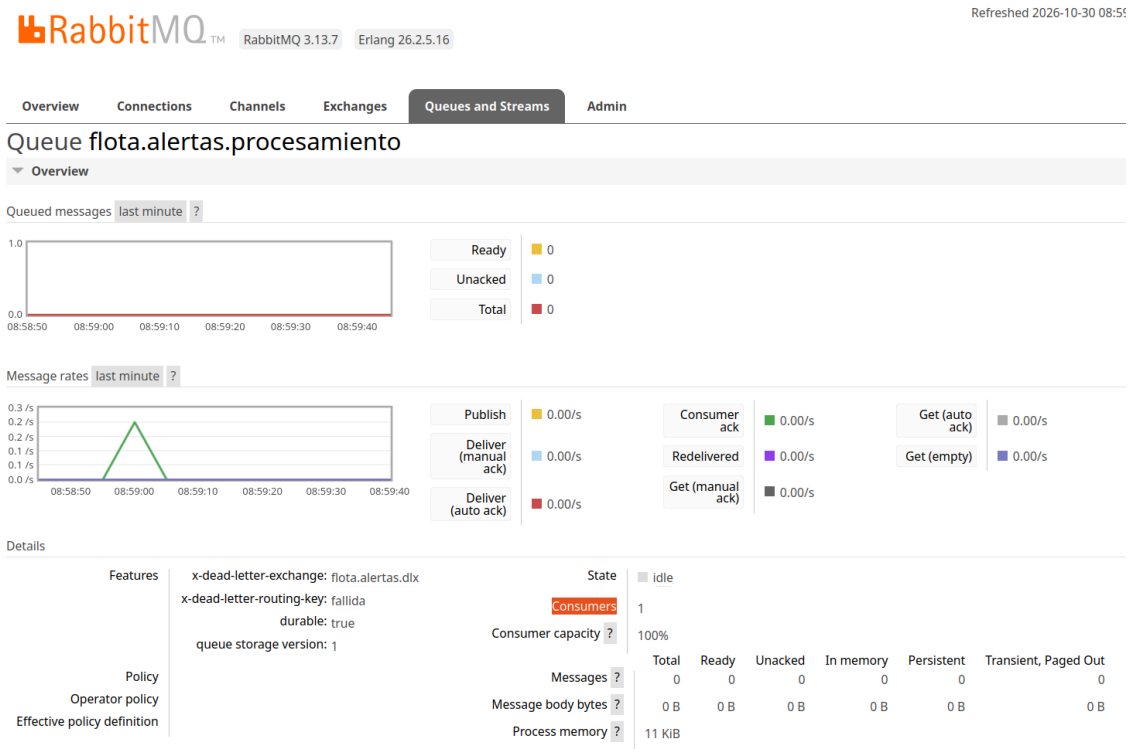


Fase 5: Orquestación Empresarial y Procesamiento de Alertas

Configuración de colas duraderas en RabbitMQ y desarrollo del microservicio consumidor que evalúa reglas de negocio sobre la cadena de frío (alertar si la temperatura > 5.0 °C).

Código de Desarrollo: Microservicio de Alertas (`procesador_alertas.py`)

```
marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯ .venv/bin/python -u procesador_alertas.py
[2026-10-30T08:38:27-05:00] Base de datos inicializada.
Regla de negocio: alertar si temperatura > 5.0 °C
[2026-10-30T08:38:27-05:00] Consumiendo cola flota.alertas.procesamiento
[2026-10-30T08:38:27-05:00] ALERTA PROCESADA [ADVERTENCIA] VH-003 (cuenca): 6.8 °C > 5.0 °C [392675bd-5cd7-4f4c-96ed-9
8ef35857b99]
[2026-10-30T08:38:27-05:00] ALERTA PROCESADA [ADVERTENCIA] VH-001 (guayaquil): 6.9 °C > 5.0 °C [2db11720-814f-4daf-9b6
c-99cead29450b]
[2026-10-30T08:38:27-05:00] ALERTA PROCESADA [CRITICA] VH-002 (quito): 8.3 °C > 5.0 °C [0c9dd7d4-7920-4b3b-ae99-282db7
f3ea21]
[2026-10-30T08:38:27-05:00] ALERTA PROCESADA [ADVERTENCIA] VH-001 (guayaquil): 5.8 °C > 5.0 °C [d4cea1d0-f5d9-4be6-8b9
4-f793ab6baad6]
[2026-10-30T08:38:27-05:00] ALERTA PROCESADA [CRITICA] VH-002 (quito): 7.8 °C > 5.0 °C [0677b937-7248-4259-a628-f29bbe
be4f24]
[2026-10-30T08:38:27-05:00] ALERTA PROCESADA [CRITICA] VH-002 (quito): 7.7 °C > 5.0 °C [a894e1da-40a5-4e58-aaea-b15005
db5a69]
```



Fase 6: Validación, Pruebas de Resiliencia y Inspección de Docker

Fase de pruebas en la que se monitorea la salud de los contenedores Docker mediante la API de RabbitMQ y comandos de inspección en terminal.

Código de Ejemplo: Verificación de Estado (`verificar_sistema.py`)

```
marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯ cd /home/marcou/Documents/SistemasDistribuidos/br
ockerMQTTdocker
.venv/bin/python verificar_sistema.py
Verificación del sistema - 2026-10-30T09:01:25-05:00
----------------------------------------------------------------------
[OK] Docker: contenedores activos
       - mosquitto_broker: Up 51 minutes (healthy) | 0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/t
cp, [::]:9001->9001/tcp
       - rabbitmq_broker: Up 51 minutes (healthy) | 4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 1
5671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
[OK] MQTT: localhost:1883 accesible
[OK] RabbitMQ: RabbitMQ 3.13.7
       - flota.alertas.procesamiento: running, 0 mensajes, 1 consumidores
       - flota.alertas.fallidas: running, 0 mensajes, 0 consumidores
[OK] SQLite: SQLite accesible
       - gps: 12 filas
       - temperatura: 12 filas
       - combustible: 12 filas
       - alertas: 8 filas
       - alertas_procesadas: 4 filas
----------------------------------------------------------------------
Estado general: SISTEMA OPERATIVO
```

```
marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯ docker compose ps
docker exec rabbitmq_broker rabbitmqctl list_queues name messages consumers durable
NAME              IMAGE                  COMMAND                  SERVICE     CREATED          STATUS
     PORTS
mosquitto_broker   eclipse-mosquitto:2.0   "/docker-entrypoint.…"   mosquitto   51 minutes ago   Up 51 minutes (healt
y)   0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp
rabbitmq_broker    rabbitmq:3-management   "docker-entrypoint.s…"   rabbitmq    51 minutes ago   Up 51 minutes (healt
y)   4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:
5672->15672/tcp, [::]:15672->15672/tcp
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages        consumers       durable
flota.alertas.fallidas  0       0       true
flota.alertas.procesamiento     0       1       true

marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯ docker exec rabbitmq_broker rabbitmqctl list_que
es name messages consumers durable
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages        consumers       durable
flota.alertas.fallidas  0       0       true
flota.alertas.procesamiento     0       1       true

marcou@marco-cst Documents/SistemasDistribuidos/brockerMQTTdocker/ ❯
```

5. Resultados

1. Broker MQTT funcional: Un broker Mosquitto corriendo en Docker, capaz de recibir y distribuir mensajes de multiples publicadores a multiples suscriptores.
2. Simulador de sensores IoT: Un script Python funcional que simula datos de telemetria de tres vehiculos (GPS, temperatura, combustible) y los publica periodicamente en topics MQTT jerarquicos.
3. Suscriptor con almacenamiento: Un suscriptor Python que recibe datos de telemetria, detecta condiciones de alerta y almacena los datos en una base de datos SQLite con tablas separadas para cada tipo de sensor.
4. Servicio puente operativo: Un servicio Python que consume datos de MQTT y los publica en colas
5. RabbitMQ, filtrando únicamente los mensajes que requieren procesamiento (alertas).

6. Preguntas de Control

- **Explique las diferencias clave entre MQTT y AMQP en términos de protocolo, patrones de comunicación y casos de uso típicos. ¿Por que MQTT es más adecuado para IoT?**

  MQTT es un protocolo ligero que utiliza el modelo publicación/suscripción mediante tópicos, mientras que AMQP ofrece mensajería más avanzada mediante exchanges y colas. MQTT es ideal para IoT porque consume menos ancho de banda y recursos, mientras que AMQP se utiliza en sistemas empresariales que requieren mayor control de mensajes.

- **¿Por que MQTT utiliza topic hierarchies (jerarquía de topics) mientras que RabbitMQ utiliza exchanges, colas y bindings? ¿Cuál es la ventaja de cada enfoque?**

  MQTT utiliza jerarquías de tópicos para organizar y filtrar mensajes de manera sencilla, facilitando la comunicación entre dispositivos IoT. RabbitMQ utiliza exchanges, colas y bindings para permitir un enrutamiento más flexible, almacenamiento de mensajes y procesamiento desacoplado

- **Describa los tres niveles de QoS en MQTT (0, 1, 2). ¿Qué ventajas y desventajas tiene cada uno? En qué escenario del monitoreo de flota utiliza cada nivel?**

  QoS 0: Entrega sin confirmación, rápida pero puede perder mensajes; ideal para datos GPS frecuentes.

  QoS 1: Garantiza al menos una entrega, pero puede generar duplicados; útil para temperatura y combustible.

  QoS 2: Garantiza una única entrega, pero consume más recursos; adecuado para alertas críticas donde se deben evitar duplicados.

7. Conclusiones

-La práctica permitió conocer el funcionamiento de una arquitectura de comunicación IoT mediante la simulación de vehículos que transmiten información a través de MQTT. La utilización de Eclipse Mosquitto en contenedores Docker facilitó la recepción y distribución de datos de ubicación, temperatura y combustible, empleando una estructura organizada de tópicos para gestionar la información de diferentes vehículos.

-Se logró establecer la comunicación entre dos sistemas de mensajería mediante el desarrollo de un intermediario en Python, encargado de transferir información desde MQTT hacia RabbitMQ utilizando AMQP. Esta integración permitió comprender cómo diferentes protocolos pueden trabajar conjuntamente, aprovechando la comunicación asíncrona y el uso de colas para administrar las alertas generadas por los vehículos.

-La incorporación de SQLite permitió conservar los registros de telemetría y consultar posteriormente la información recopilada durante la simulación. Asimismo, el procesamiento de alertas mediante condiciones de temperatura permitió identificar situaciones que requieren atención, demostrando la utilidad de integrar mecanismos de almacenamiento, validación y notificación dentro de un sistema de supervisión vehicular.